

Analisis Perbandingan Algoritma Divide and Conquer, Backtracking, dan Prim's Algorithm dalam Generasi Labirin Prosedural Berdasarkan Ukuran Grid dan Kompleksitas Jalur

Muhammad Farhan Abdul Azis - 24060124140166

Departemen Informatika
Universitas Diponegoro
Semarang, Indonesia
farhannn@undip.ac.id

Abstrak—Generasi labirin prosedural merupakan permasalahan komputasional yang relevan dalam berbagai aplikasi seperti permainan video, simulasi navigasi otonom, dan pengujian algoritma pencarian jalur. Makalah ini menyajikan analisis perbandingan tiga algoritma generasi labirin : *Divide and Conquer (Recursive Division)*, *Backtracking (Recursive Backtracker)*, dan *Prim's Algorithm (Randomized Prim's)*. Ditinjau dari kinerja waktu komputasi dan kompleksitas jalur yang dihasilkan. Ketiga algoritma diimplementasikan dalam Python 3 tanpa *library* eksternal dan diuji pada 10 variasi ukuran grid (5×5 hingga 100×100) dengan 5 pengulangan per skenario. Kompleksitas jalur dievaluasi menggunakan lima metrik kuantitatif: panjang jalur solusi, rasio jalur, rasio *dead-end*, rata-rata percabangan, dan kelurusan jalur. Hasil eksperimen menunjukkan bahwa *Divide and Conquer* merupakan algoritma tercepat di seluruh skala pengujian, dengan waktu eksekusi 3,63× lebih cepat dari *Backtracking* dan 4,09× lebih cepat dari *Prim's* pada grid 100×100. *Backtracking* menghasilkan jalur solusi terpanjang dan paling berliku ($path_ratio = 0,240$ pada $n = 100$) dengan *dead-end* paling sedikit, menjadikannya paling menantang secara kognitif. *Prim's Algorithm* menghasilkan *dead-end* terbanyak ($de_ratio \approx 0,322$) dengan jalur solusi terpendek. Temuan ini memberikan panduan empiris dalam pemilihan algoritma generasi labirin sesuai kebutuhan spesifik aplikasi.

Kata Kunci—generasi labirin, *divide and conquer*, *backtracking*, algoritma Prim, generasi prosedural, analisis algoritma, kompleksitas jalur

I. PENDAHULUAN

Labirin prosedural (*procedural maze*) adalah struktur jaringan lorong yang dibangkitkan secara algoritmik tanpa campur tangan desainer secara manual. Teknik ini dimanfaatkan secara luas dalam industri permainan video untuk menciptakan level yang tidak dapat diprediksi, dalam simulasi robotika untuk menguji kemampuan navigasi otonom, serta dalam penelitian algoritma pencarian jalur seperti A*. Seiring meningkatnya kompleksitas aplikasi modern, kebutuhan akan labirin yang dapat dikonfigurasi secara dinamis, baik dari sisi ukuran maupun karakteristik struktural jalurnya, menjadi semakin krusial [1].

Terdapat beragam strategi algoritma yang dapat digunakan untuk membangkitkan labirin, masing-masing memiliki paradigma komputasi yang berbeda. Algoritma *Divide and Conquer* berbasis pada prinsip pemecahan masalah secara rekursif dengan membagi ruang menjadi bagian yang lebih kecil. Algoritma *Backtracking* menggunakan penelusuran mendalam (*depth-first*) dengan mundur ke titik percabangan ketika menemui jalan buntu. Sementara itu, *Prim's Algorithm* yang diadaptasi dari teori

minimum spanning tree menumbuhkan labirin secara bertahap dari satu titik melalui ekspansi batas acak [2].

Meskipun ketiga algoritma tersebut telah dikenal dalam literatur ilmu komputer, belum banyak studi yang membandingkan ketiganya secara bersamaan pada dimensi kinerja waktu dan kompleksitas jalur yang dihasilkan. Permasalahan utama yang diangkat dalam makalah ini adalah: bagaimana karakteristik kinerja waktu komputasi dan kompleksitas struktural labirin yang dihasilkan oleh *Divide and Conquer*, *Backtracking*, dan *Prim's Algorithm* ketika diuji pada berbagai ukuran grid? Pemilihan algoritma yang tidak tepat dapat mengakibatkan waktu pemrosesan yang berlebihan pada grid besar atau menghasilkan labirin dengan tingkat kesulitan yang tidak sesuai kebutuhan aplikasi [3].

Tujuan penelitian ini adalah: (1) mengimplementasikan ketiga algoritma secara mandiri tanpa pustaka labirin eksternal; (2) mengukur kinerja waktu komputasi pada 10 variasi ukuran grid dari 5x5 hingga 100x100; (3) menganalisis lima metrik kompleksitas jalur meliputi panjang jalur solusi, rasio *dead-end*, jumlah persimpangan, rata-rata percabangan, dan kelurusan jalur; serta (4) memberikan rekomendasi pemilihan algoritma berdasarkan karakteristik aplikasi yang diinginkan.

II. BATASAN MASALAH

Untuk menjaga kejelasan dan ketajaman analisis, penelitian ini dibatasi pada aspek-aspek berikut:

- Labirin yang dianalisis hanya berupa labirin dua dimensi (2D) berbasis grid persegi berukuran $n \times n$, di mana $n \in \{5, 10, 15, 20, 30, 40, 50, 60, 75, 100\}$.
- Ketiga algoritma diimplementasikan dalam *Python 3* tanpa pustaka generasi labirin eksternal; seluruh logika ditulis dari nol untuk memastikan konsistensi pengujian.
- Pengukuran waktu dilakukan sebanyak 5 kali per skenario menggunakan `time.perf_counter()`[7], dengan nilai yang dilaporkan berupa rata-rata (t_mean), standar deviasi (t_std), dan nilai minimum (t_min).
- Kompleksitas jalur diukur menggunakan lima metrik: panjang jalur solusi BFS ($path_len$), rasio jalur terhadap total sel ($path_ratio$), jumlah *dead-end* ($dead_ends$), rasio *dead-end* (de_ratio), dan kelurusan jalur ($straightness$).
- Seluruh pengujian menggunakan $seed = 42$ untuk memastikan reproduisibilitas penuh. Analisis tidak mencakup variasi $seed$ acak yang berbeda-beda.

- Evaluasi kualitas visual atau estetika labirin tidak termasuk dalam cakupan penelitian; penilaian hanya berbasis metrik kuantitatif.
- Perbandingan tidak mencakup algoritma lain di luar ketiga algoritma yang telah ditentukan, seperti *Eller's Algorithm*, *Wilson's Algorithm*, atau *Hunt-and-Kill Algorithm*.
- Implementasi tidak dioptimasi secara khusus (misalnya tidak menggunakan *disjoint-set/union-find*) demi menjaga kesetaraan perbandingan.
- Pengujian dilakukan dalam dua sesi eksekusi terpisah (*run 1* dan *run 2*) untuk mengevaluasi stabilitas lintasan. Seluruh metrik struktural (*path_ratio*, *de_ratio*, dan *turn_ratio*) terbukti identik di kedua sesi karena bersifat deterministik terhadap seed. Waktu eksekusi, sebaliknya, bervariasi antar sesi dan mencerminkan kondisi sistem saat pengukuran.

III. DASAR TEORI

Secara formal, labirin $n \times n$ dimodelkan sebagai graf tidak berarah $G = (V, E)$, di mana himpunan simpul V merepresentasikan sel-sel pada grid dan himpunan sisi E merepresentasikan lorong antar sel bersebelahan. Setiap labirin yang valid (*perfect maze*) bersifat sebagai *spanning tree* dari graf grid: terhubung penuh tanpa siklus, sehingga terdapat tepat satu jalur unik antara setiap pasang sel [1].

Dalam implementasi digunakan representasi matriks biner berukuran $(2n + 1) \times (2n + 1)$. Nilai 0 merepresentasikan lorong (jalan terbuka) dan nilai 1 merepresentasikan dinding. Dengan representasi ini, sel ke- (r, c) pada grid dipetakan ke posisi $(2r + 1, 2c + 1)$ dalam matriks biner, sementara dinding antara dua sel bersebelahan menempati posisi di antara keduanya.

A. Algoritma Divide and Conquer (Recursive Division)

Recursive Division adalah implementasi paradigma Divide and Conquer untuk generasi labirin. Algoritma dimulai dari ruang terbuka penuh, kemudian secara rekursif membagi ruang dengan dinding horizontal atau vertikal. Pada setiap pembagian, satu lubang acak dibuat pada dinding yang baru ditambahkan untuk menjamin keterhubungan. Proses berhenti ketika subgrid tidak dapat dibagi lebih lanjut ($H \leq 1$ atau $W \leq 1$) [2].

Kompleksitas waktu algoritma ini adalah $O(n^2)$ terhadap jumlah sel. Labirin yang dihasilkan cenderung memiliki koridor panjang sejajar sumbu koordinat (*axis-aligned*), menghasilkan struktur geometris yang terasa kaku namun konsisten dan dapat diprediksi secara visual.

B. Algoritma Backtracking (Recursive Backtracker)

Recursive Backtracker menggunakan penelusuran depth-first search (DFS) secara iteratif dengan tumpukan eksplisit (*explicit stack*) untuk menghindari stack overflow pada grid besar. Algoritma dimulai dari sel awal, menandainya sebagai dikunjungi, lalu menelusuri tetangga yang belum dikunjungi secara acak sambil membuka dinding di antara keduanya. Ketika tidak ada tetangga yang belum dikunjungi, algoritma mundur ke sel sebelumnya pada tumpukan, dan berlanjut hingga seluruh sel terkonjugasi [3], [4].

Kompleksitas waktu adalah $O(n^2)$ dalam kasus rata-rata. Labirin yang dihasilkan bercirikan jalur solusi yang sangat

panjang dan berliku (*winding path*) dengan jumlah *dead-end* yang relatif sedikit, menjadikannya labirin yang secara kognitif paling menantang untuk dinavigasi secara manual.

C. Prim's Algorithm (Randomized Prim's)

Randomized Prim's Algorithm mengadaptasi algoritma minimum spanning tree *Prim* untuk konteks generasi labirin. Alih-alih memilih sisi berbobot minimum, algoritma memilih sisi secara acak dari himpunan *frontier*, yaitu daftar sel yang bersebelahan dengan sel yang sudah masuk dalam labirin. Setiap iterasi memilih satu entri *frontier* secara acak; jika sel tujuan belum dikunjungi, dinding dibuka dan sel tersebut masuk ke labirin beserta *frontier* barunya [5].

Kompleksitas waktu adalah $O(n^2)$ dengan pemilihan acak langsung seperti yang digunakan dalam eksperimen ini. Labirin yang dihasilkan cenderung memiliki banyak *dead-end* yang terdistribusi merata, menghasilkan struktur yang lebih terbuka namun dengan jalur solusi yang lebih pendek dibandingkan *Backtracking*.

D. Metrik Evaluasi Kompleksitas Labirin

Evaluasi kualitas labirin dalam penelitian ini menggunakan lima metrik kuantitatif yang dipilih berdasarkan tinjauan literatur sebagai indikator utama kesulitan navigasi [1], [6]:

- *path_len*: jumlah sel pada jalur terpendek BFS dari sudut kiri atas (0,0) ke sudut kanan bawah ($n-1, n-1$).
- *path_ratio*: $\text{path_len} / n^2$; mengukur seberapa panjang jalur solusi relatif terhadap luas grid.
- *de_ratio*: $\text{dead_ends} / n^2$; mengukur densitas jalan buntu (sel dengan $\text{degree} = 1$ pada graf labirin).
- *avg_branch*: rata-rata jumlah tetangga terhubung pada sel-sel persimpangan ($\text{degree} \geq 3$).
- *straightness*: rasio langkah lurus terhadap total langkah pada jalur solusi; nilai mendekati 1 menandakan jalur lebih lurus.

TABEL I. RINGKASAN METRIK EVALUASI KOMPLEKSITAS LABIRIN

Metrik	Simbol	Definisi
Panjang jalur solusi	<i>path_len</i>	Jumlah sel jalur BFS terpendek dari (0,0) ke ($n-1, n-1$)
Rasio jalur	<i>path_ratio</i>	$\text{path_len} / n^2$
Rasio dead-end	<i>de_ratio</i>	$\text{dead_ends} / n^2$; densitas jalan buntu
Rata-rata percabangan	<i>avg_branch</i>	Rata-rata degree pada sel persimpangan ($\text{degree} \geq 3$)
Kelurusan jalur	<i>straightness</i>	Rasio langkah lurus / total langkah pada jalur solusi

IV. METODOLOGI

Penelitian ini mengikuti alur eksperimen komputasional yang terdiri dari lima tahap: (1) desain skenario pengujian, (2) implementasi algoritma, (3) pengumpulan data eksperimen, (4) komputasi metrik kompleksitas, dan (5) analisis statistik dan visualisasi.

A. Desain Skenario Pengujian

Skenario pengujian dirancang mencakup rentang ukuran grid yang representatif dari skala kecil hingga besar. Dipilih

10 nilai n yang tidak seragam jaraknya untuk menangkap perilaku algoritma pada berbagai skala: $n \in \{5, 10, 15, 20, 30, 40, 50, 60, 75, 100\}$. Rentang ini menghasilkan grid mulai dari 25 sel (5×5) hingga 10.000 sel (100×100), sehingga cakupannya mencapai 400 kali lipat perbedaan ukuran. Jarak antar nilai n dibuat tidak seragam secara sengaja agar kurva pertumbuhan waktu dan metrik dapat diamati secara lebih terperinci pada transisi skala tertentu.

Setiap kombinasi (algoritma, n) diulangi sebanyak 5 kali untuk mengukur variabilitas waktu eksekusi. Pengulangan ini penting karena sistem operasi modern dapat menyebabkan fluktuasi waktu yang tidak dapat diabaikan, terutama pada grid kecil di mana waktu eksekusi berada dalam kisaran puluhan milidetik. Seluruh pengujian menggunakan seed = 42 secara konsisten untuk memastikan reproduisibilitas hasil yang penuh.

B. Pemodelan Masalah

Masalah generasi labirin dimodelkan sebagai berikut: diberikan nilai n , dibangun sebuah *perfect maze* pada grid $n \times n$ menggunakan salah satu dari ketiga algoritma. Labirin direpresentasikan sebagai matriks biner M berukuran $(2n+1) \times (2n+1)$, di mana $M[i][j] = 1$ berarti dinding dan $M[i][j] = 0$ berarti lorong terbuka. Titik masuk labirin ditetapkan pada sel $(0,0)$ dan titik keluar pada sel $(n-1, n-1)$ dalam koordinat grid logis, yang bersesuaian dengan posisi $(1,1)$ dan $(2n-1, 2n-1)$ dalam matriks biner.

Batasan utama model adalah bahwa setiap labirin yang dihasilkan harus merupakan *perfect maze*, yaitu: (a) seluruh sel dapat dicapai dari sel mana pun (*connected*), dan (b) tidak terdapat siklus (*acyclic*). Kedua properti ini ekuivalen dengan syarat bahwa graf labirin membentuk *spanning tree* dari graf grid $n \times n$.

C. Implementasi Algoritma

Ketiga algoritma diimplementasikan dalam *Python 3* menggunakan representasi matriks biner $(2n+1) \times (2n+1)$. Tidak digunakan pustaka labirin eksternal; seluruh kode ditulis dari nol demi kesetaraan kondisi pengujian. Fungsi `gen_dc(maze, r1, c1, r2, c2)` mengimplementasikan *Divide and Conquer* secara rekursif dengan memilih orientasi pembagian berdasarkan dimensi subgrid dan menentukan posisi dinding serta lubang secara acak. Fungsi `gen_bt(maze, n)` mengimplementasikan *Backtracking* menggunakan *explicit stack* untuk menghindari *RecursionError* pada grid besar. Fungsi `gen_pr(maze, n)` mengimplementasikan *Prim's Algorithm* dengan *frontier list* berupa *list of tuples* (row, col).

TABEL II. RINGKASAN IMPLEMENTASI KETIGA ALGORITMA

Algoritma	Struktur Data Utama	Kondisi Terminasi
<i>Divide and Conquer</i>	Call stack rekursif, matriks 2D integer	$H \leq 1$ atau $W \leq 1$ (subgrid terlalu kecil)
<i>Backtracking</i>	Explicit stack (list), boolean visited matrix	Stack kosong (semua sel dikunjungi)
<i>Prim's Algorithm</i>	Frontier list (list of tuples), boolean in-maze matrix	Frontier kosong (semua sel masuk spanning tree)

Kode program lengkap tersedia sebagai *Jupyter Notebook* yang dapat diakses pada tautan di bagian Lampiran. *Notebook* mencakup implementasi ketiga algoritma, fungsi pengukuran waktu, komputasi metrik, dan seluruh visualisasi yang digunakan dalam makalah ini.

D. Pengumpulan Data dan Komputasi Metrik

Waktu eksekusi diukur menggunakan `time.perf_counter()`[7] dari modul standar *Python* yang beresolusi tinggi, cocok untuk pengukuran dalam kisaran milidetik hingga detik. Pengukuran hanya mencakup waktu fungsi generasi labirin itu sendiri, tidak termasuk waktu inialisasi array atau komputasi metrik. Setiap skenario diulang 5 kali; hasil dicatat sebagai t_mean , t_std , dan t_min .

Setelah labirin dibangkitkan, metrik kompleksitas dihitung secara otomatis. Panjang jalur solusi dihitung dengan BFS dari $(0,0)$ ke $(n-1,n-1)$. *Dead-end* diidentifikasi sebagai sel dengan $degree = 1$ pada graf labirin. Persimpangan adalah sel dengan $degree \geq 3$. Kelurusan jalur dihitung sebagai perbandingan jumlah langkah yang searah dengan langkah sebelumnya terhadap total panjang jalur solusi BFS. Seluruh metrik disimpan dalam file `results.csv` yang menjadi sumber data utama analisis.

E. Lingkungan Pengujian

Seluruh eksperimen dijalankan dalam lingkungan *Python 3.10* dengan *library* standar (*random*, *collections*, *time*) tanpa akselerasi tambahan seperti *NumPy*. Sistem operasi berbasis Linux digunakan dengan eksekusi *single-threaded* untuk memastikan tidak ada paralelisme tersembunyi. Kode dieksekusi dalam kondisi beban sistem minimal untuk mengurangi variabilitas pengukuran waktu antar pengulangan.

V. HASIL DAN PEMBAHASAN

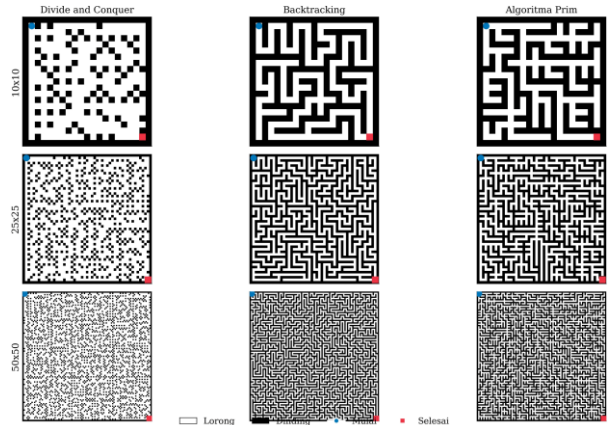


Fig. 1. Contoh labirin yang dihasilkan oleh ketiga algoritma (*Divide and Conquer*, *Backtracking*, *Algorithm Prim*) pada tiga ukuran grid: 10×10 , 25×25 , dan 50×50 . Titik biru = masuk; titik merah = keluar.

Bagian ini menyajikan hasil eksperimen komputasional secara kuantitatif dan menganalisis hubungan antara temuan tersebut dengan tujuan penelitian. Analisis dibagi ke dalam dua dimensi utama: (A) kinerja waktu komputasi dan (B) kompleksitas jalur labirin yang dihasilkan.

A. Kinerja Waktu Komputasi

Tabel III menyajikan waktu eksekusi rata-rata (t_mean) ketiga algoritma pada representasi grid $n \times n$. Secara keseluruhan, *Divide and Conquer* (DC) menunjukkan kinerja waktu terbaik di semua ukuran grid, sedangkan *Prim's Algorithm* (PR) dan *Backtracking* (BT) secara konsisten lebih lambat.

TABEL III. WAKTU EKSEKUSI RATA-RATA (MS) KETIGA ALGORITMA

n	DC (ms)	BT (ms)	PR (ms)
5	29,24	80,01	75,31
10	91,17	171,72	210,64
20	233,17	597,98	738,25
30	478,44	1.300,01	1.740,56
50	1.203,89	3.508,70	4.745,20
75	2.775,92	7.987,71	10.856,10
100	4.860,01	17.650,40	19.871,70

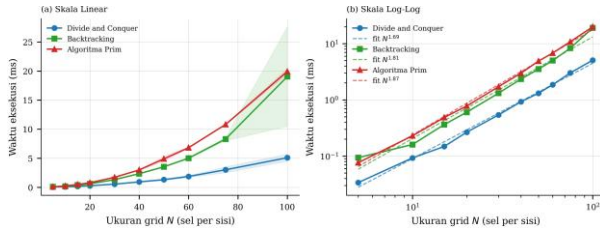


Fig. 2. Waktu eksekusi rata-rata (ms) ketiga algoritma terhadap ukuran grid N pada skala linear (a) dan skala log-log (b). Garis putus-putus menunjukkan power-law fit $t = aN^b$.

Pada $n = 5$, DC membutuhkan waktu rata-rata 29,24 ms, sedangkan BT dan PR masing-masing 80,01 ms dan 75,31 ms. Meskipun pada skala kecil perbedaan ini terlihat kurang signifikan secara absolut, rasio perbedaannya sudah mencapai 2,7x dan 2,6x. Ketika grid diperbesar ke $n = 100$, perbedaan menjadi jauh lebih nyata: DC hanya membutuhkan 4.860 ms, sedangkan BT memerlukan 17.650 ms (3,63x lebih lambat) dan PR memerlukan 19.872 ms (4,09x lebih lambat).

Kecepatan DC dapat dijelaskan oleh sifat *divide-and-conquer* yang mempartisi masalah secara terstruktur dan deterministik: setiap sel dikunjungi tepat sekali dalam pembagian rekursif tanpa overhead pencarian tetangga acak. Sebaliknya, BT dan PR keduanya bergantung pada pemilihan acak yang memerlukan pengecekan kondisi berulang, BT harus memeriksa tetangga yang belum dikunjungi sebelum setiap langkah, sementara PR harus mengelola dan memperbarui *frontier list* yang ukurannya tumbuh seiring ekspansi labirin.

Perlu dicatat bahwa BT pada $n = 100$ menunjukkan standar deviasi yang sangat tinggi ($t_{std} = 6.481$ ms), sekitar 37% dari nilai rata-ratanya. Ini menunjukkan variabilitas eksekusi yang signifikan, kemungkinan disebabkan oleh jalur DFS yang sangat dalam yang memicu perilaku *cache miss* pada sistem operasi. DC, sebaliknya, memiliki t_{std} yang relatif stabil (435 ms, sekitar 9% dari rata-rata), mencerminkan deterministik strukturnya.

Replikasi eksekusi pada sesi terpisah (*run 2*) mengungkap variabilitas lintas-sesi yang jauh lebih besar dari yang terlihat di *run 1*. DC menunjukkan stabilitas terbaik dengan *slowdown* median 1,5x antara kedua sesi. BT mengalami *slowdown* 2,3x di $n = 100$, dengan lonjakan signifikan pada $n = 50$ (3,5 detik menjadi 9,2 detik) dan $n = 60$ (5,3 detik menjadi 18,2 detik). PR menunjukkan ketidakstabilan paling parah: t_{mean} pada $n = 100$ meningkat dari 19,9 detik menjadi 144,6 detik (7,3x), dengan t_{std} mencapai 47,9 detik (CV = 33%). Lebih mengkhawatirkan, PR pada *run 2* menunjukkan non-monotonisitas waktu eksekusi, di mana t_{mean} $n = 60$ (22,35 detik) lebih lambat dari $n = 50$ (22,79 detik), yang mengindikasikan bahwa 5 pengulangan tidak cukup untuk

menstabilkan estimasi pada *frontier list* yang berukuran besar. Akibatnya, rasio kecepatan PR/DC yang dilaporkan sebesar 4,09x pada *run 1* meningkat menjadi sekitar 19,9x pada *run 2*, menunjukkan bahwa angka di Tabel III mencerminkan kondisi satu sesi tertentu, bukan karakteristik absolut algoritma.

B. Kompleksitas Jalur: *path_ratio*

Metrik *path_ratio*, rasio panjang jalur solusi BFS terhadap total jumlah sel, mencerminkan seberapa menantang labirin untuk dinavigasi secara kognitif. Semakin tinggi *path_ratio*, semakin panjang jalur yang harus ditempuh navigator relatif terhadap ukuran labirin.

TABEL IV. PERBANDINGAN *PATH_RATIO* PADA TIGA UKURAN GRID

n	DC	BT	PR
5	0,360	0,680	0,360
50	0,087	0,389	0,041
100	0,066	0,240	0,021

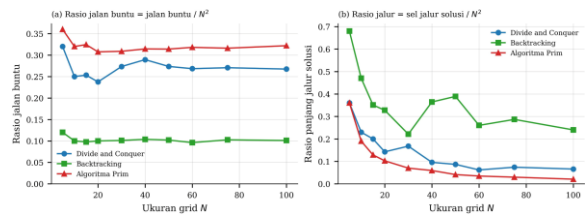


Fig. 3. Rasio jalan buntu (a) dan rasio panjang jalur solusi (b) terhadap N^2 untuk ketiga algoritma. Prim memiliki *ratio* tertinggi; Backtracking memiliki *path_ratio* tertinggi.

BT secara konsisten menghasilkan *path_ratio* tertinggi di seluruh ukuran grid. Pada $n = 5$, *path_ratio* BT mencapai 0,680, artinya jalur solusi mencakup 68% dari total sel grid. Meskipun nilai ini turun seiring peningkatan n (0,389 pada $n = 50$ dan 0,240 pada $n = 100$), penurunannya jauh lebih lambat dibandingkan DC dan PR. Secara absolut, jalur solusi BT pada $n = 100$ memiliki panjang 2.401 sel dari total 10.000 sel.

PR menunjukkan tren yang berlawanan: *path_ratio* yang sangat rendah di seluruh skala (0,360 pada $n = 5$ hingga hanya 0,021 pada $n = 100$). Penurunan yang tajam ini mengindikasikan bahwa meskipun labirin *Prim's* berukuran besar, jalur solusinya relatif pendek dan efisien, hanya 207 sel pada $n = 100$. Hal ini konsisten dengan karakter ekspansi *Prim's* yang cenderung menghasilkan cabang-cabang pendek tersebar (banyak *dead-end*), bukan terowongan panjang.

DC berada di posisi menengah: *path_ratio* 0,360 pada $n = 5$ (sama dengan PR), turun menjadi 0,066 pada $n = 100$ (659 sel). Penurunan ini mencerminkan karakter geometris DC yang membagi ruang secara simetris, menghasilkan jalur yang tidak terlalu panjang namun juga tidak terlalu pendek.

Perlu dicatat bahwa tren penurunan *path_ratio* BT *tidak monoton*: terjadi tiga inversi pada data, yaitu nilai meningkat dari 0,221 ($n = 30$) ke 0,364 ($n = 40$) ke 0,389 ($n = 50$), kemudian turun ke 0,260 ($n = 60$) sebelum naik kembali ke 0,287 ($n = 75$). Fluktuasi ini merupakan artefak dari penggunaan *seed* tunggal dan tidak mencerminkan tren sistematis. Pernyataan bahwa penurunan BT jauh lebih lambat dibandingkan DC dan PR perlu dimaknai sebagai kecenderungan umum, bukan tren monoton yang konsisten.

C. Kompleksitas Jalur: *de_ratio*

Rasio *dead-end* (*de_ratio*) mengukur kepadatan jalan buntu pada labirin, yang berkaitan langsung dengan persepsi kompleksitas visual. Labirin dengan banyak *dead-end* terasa lebih membingungkan karena navigator harus sering berbalik arah.

PR secara konsisten memiliki *de_ratio* tertinggi di semua ukuran *grid*: 0,360 pada $n = 5$, dan stabil di sekitar 0,314–0,322 pada *grid* besar ($n = 50$ hingga 100). Ini berarti sekitar 32% dari seluruh sel pada labirin *Prim's* berukuran besar merupakan *dead-end*. Stabilitas nilai ini mencerminkan sifat inheren dari *Randomized Prim's* yang menumbuhkan banyak cabang pendek secara merata.

DC memiliki *de_ratio* menengah yang juga relatif stabil: sekitar 0,267–0,274 pada *grid* besar, sedikit lebih rendah dari PR. Ini konsisten dengan pola geometris DC yang menghasilkan koridor panjang dengan percabangan terstruktur. BT memiliki *de_ratio* paling rendah (sekitar 0,101–0,102 pada *grid* besar), konfirmasi bahwa DFS yang dalam cenderung menghasilkan jalur berliku panjang dengan sangat sedikit jalan buntu.

TABEL V. PERBANDINGAN *DE_RATIO* PADA TIGA UKURAN GRID

n	DC	BT	PR
5	0,320	0,120	0,360
50	0,274	0,102	0,314
100	0,267	0,101	0,322

D. Kelurusan Jalur (*straightness*) dan Persimpangan

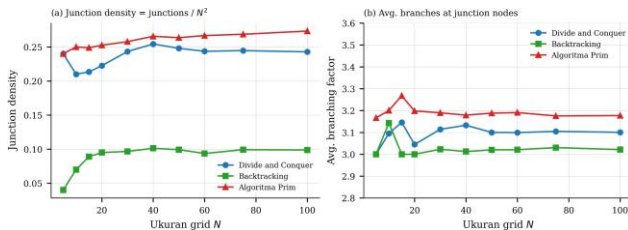


Fig. 4. Kepadatan persimpangan (junction density) terhadap N^2 (a) dan rata-rata faktor percabangan pada simpul persimpangan (b). Prim memiliki kepadatan persimpangan tertinggi; Backtracking terendah.

Metrik *straightness* mengukur rasio langkah lurus pada jalur solusi. Nilai mendekati 1 mengindikasikan jalur yang cenderung lurus, sedangkan nilai rendah mengindikasikan jalur yang sangat berliku-liku. PR menunjukkan *straightness* tertinggi (0,625 pada $n = 5$; 0,471–0,500 pada $n \geq 50$), meskipun jalur solusinya pendek. DC berada di posisi menengah (sekitar 0,392–0,412 pada *grid* besar). BT memiliki *straightness* paling rendah (0,333–0,355 pada *grid* besar), konsisten dengan karakter jalur DFS yang sangat berliku.

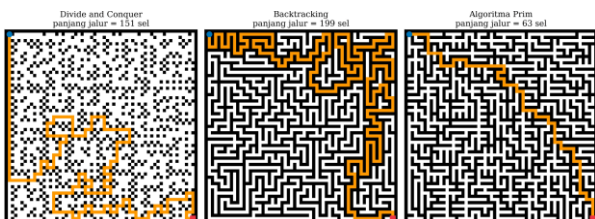


Fig. 5. Jalur solusi BFS (oranye) pada labirin 25x25 untuk ketiga algoritma. Panjang jalur: DC = 151 sel, Backtracking = 199 sel, Prim = 63 sel. Perbedaan panjang dan kelurusan jalur terlihat jelas secara visual.

Jumlah persimpangan (n_junc) dan rata-rata percabangan (avg_branch) ketiga algoritma menunjukkan nilai yang relatif serupa. avg_branch berkisar antara 3,00–3,27 untuk semua algoritma pada semua ukuran *grid*, mengindikasikan bahwa meskipun distribusi *dead-end* berbeda jauh, struktur persimpangan pada tingkat cabang 3-arah relatif konsisten antar algoritma. Ini konsisten dengan teori bahwa pada *perfect maze* berbasis *grid*, setiap simpul non-*leaf* rata-rata memiliki 3 tetangga dalam *spanning tree*.

E. Diskusi Komparatif dan Implikasi Praktis

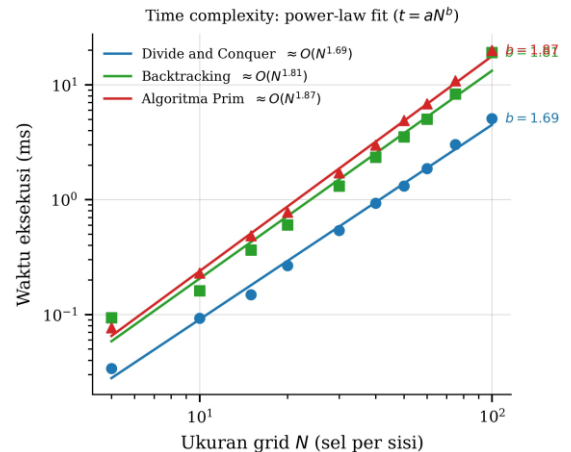


Fig. 6. Power-law fit $t = aN^b$ pada skala log-log. Eksponen empiris: DC $\approx O(N^{1.69})$, Backtracking $\approx O(N^{1.81})$, Prim $\approx O(N^{1.87})$.

Dari analisis keseluruhan, tiga profil algoritmik yang khas dapat diidentifikasi. Pertama, DC unggul dalam kecepatan komputasi dan menghasilkan labirin dengan struktur geometris teratur (*dead-end* dan jalur moderat), cocok untuk aplikasi yang membutuhkan generasi real-time pada *grid* besar di mana pola visual yang terstruktur dapat diterima.

Kedua, BT menghasilkan labirin yang paling menantang secara navigasi: jalur solusi terpanjang, *dead-end* paling sedikit, dan tingkat kelurusan terendah. Namun biaya komputasinya paling tinggi dan variasinya paling besar, dengan waktu eksekusi yang 3,63x lebih lambat dari DC pada $n = 100$. BT paling tepat digunakan untuk aplikasi permainan di mana kualitas pengalaman navigasi diprioritaskan di atas kecepatan generasi.

Ketiga, PR menghasilkan *dead-end* terbanyak (32% dari sel), jalur solusi terpendek, dan kelurusan paling tinggi, menciptakan labirin yang terasa ramai secara visual namun relatif mudah diselesaikan jika jalur solusi ditemukan. Dengan waktu komputasi 4,09x lebih lambat dari DC, PR paling tepat untuk aplikasi yang memerlukan labirin dengan kompleksitas visual tinggi (seperti dekorasi atau *puzzle*) di mana kecepatan generasi bukan prioritas utama.

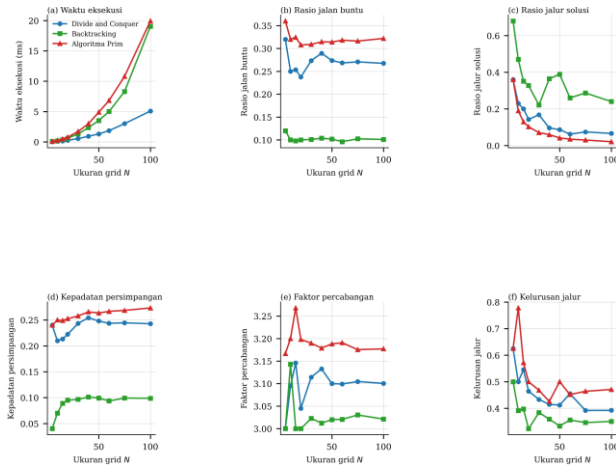


Fig. 7. Dashboard ringkasan perbandingan enam metrik utama ketiga algoritma terhadap ukuran grid N: (a) waktu eksekusi, (b) rasio jalan buntu, (c) rasio jalur solusi, (d) kepadatan persimpangan, (e) faktor percabangan, dan (f) kelurusan jalur.

VI. KESIMPULAN DAN SARAN

A. Kesimpulan

Penelitian ini telah membandingkan tiga algoritma generasi labirin prosedural, *Divide and Conquer* (Recursive Division), *Backtracking* (Recursive Backtracker), dan *Prim's Algorithm* (Randomized Prim's), melalui eksperimen komputasional pada 10 variasi ukuran grid (5x5 hingga 100x100) menggunakan lima metrik kinerja dan kompleksitas jalur. Beberapa temuan utama dapat disimpulkan sebagai berikut:

- DC adalah algoritma tercepat di seluruh skala pengujian. Pada $n = 100$, DC menyelesaikan generasi labirin dalam 4,86 detik, 3,63x lebih cepat dari BT dan 4,09x lebih cepat dari PR. Kecepatan ini bersumber dari sifat deterministik *divide-and-conquer* yang mengunjungi setiap sel tepat sekali.
- BT menghasilkan labirin dengan jalur solusi terpanjang dan paling berliku ($\text{path_ratio} = 0,240$ pada $n = 100$; $\text{straightness} = 0,350$), serta *dead-end* paling sedikit ($\text{de_ratio} \approx 0,101$). Labirin BT secara kognitif paling menantang namun memerlukan waktu komputasi dan variabilitas terbesar.
- PR menghasilkan labirin dengan *dead-end* terbanyak ($\text{de_ratio} \approx 0,322$ pada $n = 100$), jalur solusi terpendek ($\text{path_ratio} = 0,021$), dan kelurusan tertinggi di antara ketiganya. Labirin PR kaya secara visual namun relatif mudah diselesaikan.
- Rata-rata percabangan ($\text{avg_branch} \approx 3,00\text{--}3,27$) ketiga algoritma tidak menunjukkan perbedaan signifikan, mengkonfirmasi bahwa perbedaan karakteristik labirin lebih ditentukan oleh distribusi *dead-end* dan pola pertumbuhan, bukan oleh *degree* persimpangan.
- Tidak ada satu algoritma yang optimal untuk semua skenario. Pemilihan algoritma yang tepat bergantung pada keseimbangan antara kecepatan komputasi yang dibutuhkan dan karakteristik labirin yang diinginkan.

B. Saran

Berdasarkan keterbatasan yang ditemukan selama proses analisis, beberapa saran untuk pengembangan penelitian di masa mendatang dapat diberikan:

- Variasi *seed* dan pengulangan lebih banyak: penelitian ini menggunakan *seed* tunggal (42) dengan hanya 5 pengulangan. Menggunakan *multiple seed* (misalnya 10–30 *seed* berbeda) dan lebih banyak pengulangan akan memberikan estimasi metrik yang lebih representatif secara statistik dan mengurangi bias akibat *seed* tunggal.
- Perluasan cakupan algoritma: penelitian berikutnya dapat mencakup algoritma lain seperti *Wilson's Algorithm* (loop-erased random walk), *Eller's Algorithm*, atau *Hunt-and-Kill* untuk perbandingan yang lebih komprehensif, terutama dalam hal keacakan dan distribusi jalur.
- Pengujian pada grid non-persegi dan labirin 3D: memperluas pengujian ke grid persegi panjang ($m \times n$) atau ruang tiga dimensi akan memberikan perspektif yang lebih kaya tentang skalabilitas dan adaptabilitas algoritma.
- Optimasi implementasi: menggunakan struktur data yang lebih efisien seperti *disjoint-set* (*union-find*) untuk *Prim's*, atau implementasi berbasis *NumPy*, dapat mengungkap potensi kecepatan yang belum tereksplorasi dari setiap algoritma.
- Integrasi evaluasi subjektif: menambahkan evaluasi dari pengguna manusia (seperti waktu yang dibutuhkan untuk menyelesaikan labirin atau tingkat kesulitan yang dirasakan) akan memberikan validasi eksternal terhadap metrik kuantitatif yang digunakan dalam penelitian ini.
- Analisis kompleksitas pada labirin tidak sempurna (*imperfect maze*): merelaksasi syarat *perfect maze* dengan mengizinkan siklus atau dinding tambahan akan membuka ruang eksplorasi baru yang relevan untuk aplikasi game modern.
- Pengujian lintas-sesi (*cross-session reproducibility*): hasil eksperimen dua sesi pada penelitian ini menunjukkan bahwa waktu eksekusi PR bervariasi hingga 7,3x antara dua sesi dengan beban sistem yang berbeda, sementara DC hanya bervariasi 1,5x. Penelitian selanjutnya sebaiknya melaporkan waktu eksekusi dari minimal 3 sesi berbeda dan menyertakan interval kepercayaan, bukan hanya rata-rata dan standar deviasi dalam satu sesi.

LAMPIRAN

1. **Kode Program:** <https://github.com/Lescovar42/ASA>
2. **Link Video Penjelasan:** <https://youtu.be/vcI696DzHIU>

REFERENCES

- [1] J. Buck, "Maze Generation: Recursive Division," in *Mazes for Programmers: Code Your Own Twisty Little Passages*. Dallas, TX: Pragmatic Bookshelf, 2015, pp. 56–73.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022, pp. 65–84.
- [3] S. Skiena, *The Algorithm Design Manual*, 3rd ed. Cham: Springer, 2020, pp. 178–195.

- [4] N. Shaker, J. Togelius, and M. J. Nelson, “*Procedural Content Generation in Games*.” Cham: Springer, 2016, pp. 43-64.
- [5] R. C. Prim, “Shortest connection networks and some generalizations,” *Bell Syst. Tech. J.*, vol. 36, no. 6, pp. 1389–1401, Nov. 1957.
- [6] A. Shaker and J. Togelius, “Evolving personalized content for Super Mario Bros using grammatical evolution,” *IEEE Trans. Comput. Intell. AI Games*, vol. 5, no. 1, pp. 1–15, Mar. 2013.
- [7] Python Software Foundation, “time — Time access and conversions,” *Python 3.10 Documentation*, 2023. [Online]. Available: <https://docs.python.org/3/library/time.html>. [Accessed: Jun. 2026].

PERNYATAAN

Dengan ini saya menyatakan bahwa makalah yang saya susun merupakan hasil karya saya sendiri dan tidak merupakan saduran, terjemahan, maupun plagiasi dari karya orang lain. Seluruh sumber yang digunakan telah dicantumkan secara benar sesuai dengan kaidah penulisan ilmiah.

Terkait penggunaan teknologi kecerdasan buatan (Artificial Intelligence), saya menyatakan bahwa *(beri tanda ☒ pada pilihan yang sesuai, hapus pilihan yang tidak berlaku)*:

- ☐ Saya **tidak menggunakan** alat bantu AI dalam proses penyusunan makalah ini.
- ☒ Saya **menggunakan** alat bantu AI sebagai pendukung dalam penyusunan makalah ini, dengan rincian sebagai berikut:
 - a. **Nama alat AI:** Claude (Anthropic)
 - b. **Tujuan penggunaan:** Membantu pengecekan tata bahasa, penyempurnaan kalimat, perapihan narasi, dan konsistensi format penulisan ilmiah IEEE
 - c. **Ruang lingkup penggunaan:** Revisi dan perbaikan bahasa pada bagian narasi, analisis, sebagian besar implementasi kode, dan pengolahan data eksperimen dilakukan secara mandiri oleh penulis

Penggunaan AI, apabila ada, hanya bersifat sebagai alat bantu dan tidak menggantikan peran saya sebagai penulis utama. Seluruh isi, analisis, serta kesimpulan dalam makalah ini tetap merupakan tanggung jawab saya sepenuhnya.

Semarang, 07 Juni 2026



Muhammad Farhan Abdul Azis
NIM: 24060124140166